目录

第1章 数据结构 1
1.1 树状数组 1
1.1.1 离线_统计q个区间元素种数 1
1.1.2 权值树状数组 + 离散化 3
1.1.3 树状数组 5
1.2 线段树 6
1.2.1 线段树 6
1.2.2 加 + 乘 8
1.2.3 修改 + 加 10
1.2.4 最大子段和 13
1.2.5 分治 + 多个信息 15
1.2.6 权值线段树 + 离散化 18
1.2.7 区间开更号（剪枝） 21
1.2.8 方差 23
1.2.9 GCD 26
1.3 ST表 29
1.3.1 尾插操作 29
1.3.2 ST表 31
第2章 动态规划 32
2.1 树形dp 32
2.1.1基本树形dp 32
2.1.1.1 最大独立集 32
2.1.1.2 最小边覆盖 34
2.1.1.3 最小点覆盖 34
2.1.2 重心与直径 34
2.1.2.1 重心 34
2.1.2.2带点权边权重心 39
2.1.2.3 带点权重心 34
2.2 LCA 43
2.2.1 倍增法 43
2.2.2 Tarjan 45
2.2.3 树上两点距离 47
2.2.4 树上询问 49
2.3 数位dp 34
2.3.1 萌数 52
2.3.2 数位dp(枚举区间不降数) 54
2.4 状压dp 34
2.4.1 棋盘覆盖3 56
2.4.2 TSP 58

第3章 图论 59
3.1 01bfs 59
3.2 dijkstra 60
第4章 数学知识 61
4.1 数论 34
4.1.1 分解质因子 61
4.1.2 EXGCD 62
4.2 线性代数 63
4.2.1矩阵快速幂 63
第5章 搜索 65
5.1 迭代加深 65
5.2 双向广搜_八数码 67
5.3 折半搜索 69
终章 附录 71
关同步 71
快速读写 72
快速幂 73
龟速乘 74
离散化（二分） 75
GCD 76
枚举子状态 76

第1章 数据结构

树状数组

离线_统计q个区间元素种数

```
#include <bits/stdc++.h>
using namespace std;

#define lowbit(x) (x & -x)

const int N = 1e6 + 10;
int a[N], n, m;
struct node {
        int id;
        int l, r;
}q[N];

int ret[N]; // ret[k] : 第 k 个问题答案

int mp[N]; // mp[k] : k 上一次出现的位置

int s[N];

// s[x] += k
void modify(int x, int k) {
        for (int i = x; i <= n; i += lowbit(i)) s[i] += k;
}

// return [1, x]
int query(int x) {
        int ret = 0;
        for (int i = x; i; i -= lowbit(i)) ret += s[i];
        return ret;
}

// 贡献法
// 算法原理概述：对于某一个元素 x, 只点亮最右边的 x
// 总述： 以cur为右端点的区间一定正确
```

```cpp
// 1. 对于 x 最多贡献一次，因为在[1, cur] 之间只保留了 1 个x
// 2. x 不可能不贡献，因为保留的是最右边的 x, 如果[l, cur] 之间有 x, 一定会包含被保留的 x
// 3. 所以对于某一个特定的 x ,贡献且仅贡献一次
void solve() {
        // 优先解决 r 较小的问题
        sort(q + 1, q + m + 1, [] (node l, node r) { return l.r < r.r; } );

        //cur : 将要考虑的元素
        for (int i = 1, cur = 1; i <= m; ++i) {
                auto [id, l, r] = q[i];
                while (cur <= r) {
                        int x = a[cur];
                        int pre = mp[x];

                        if (pre) modify(pre, -1); // 消除前面的
                        modify(cur, 1); // 统计现在的

                        mp[x] = cur;
                        ++cur;
                }
                ret[id] = query(r) - query(l - 1);
        }
}

int main() {
        cin >> n;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        cin >> m;
        for (int i = 1; i <= m; ++i) {
                auto& [id, l, r] = q[i];
                id = i; cin >> l >> r;
        }

        solve();

        for (int i = 1; i <= m; ++i) cout << ret[i] << endl;
        cout << endl;

        return 0;
}
```

权值树状数组 + 离散化

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lowbit(x) (x & -x)

const int N = 1e6 + 10;

int disc[N], pos;
LL s[N];
int a[N], n;

int id(int x) {
        int l = 1, r = pos;
        while (l < r) {
                int m = l + r >> 1;
                if (disc[m] >= x) r = m;
                else l = m + 1;
        }
        return l;
}

void modify(int p, int x) {
        for (int i = p; i <= pos; i += lowbit(i)) s[i] += x;
}

LL query(int x) {
        LL ret = 0;
        for (int i = x; i; i -= lowbit(i)) ret += s[i];
        return ret;
}

int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);

        cin >> n;
        for (int i = 1; i <= n; ++i) {
                int x; cin >> x;
                a[i] = x; disc[++pos] = x;
        }
```

```
        sort(disc + 1, disc + pos + 1);
        pos = unique(disc + 1, disc + pos + 1) - (disc + 1);

        LL ret = 0;
        for (int i = 1; i <= n; ++i) {
                int k = id(a[i]);
                ret += query(pos) - query(k);
                modify(k, 1);
        }
        cout << ret << endl;

        return 0;
}
```

树状数组

```cpp
#include <bits/stdc++.h>
using namespace std;
using LL = long long;
#define lowbit(x) (x & -x)

const int N = 1e6 + 10, INF = 0x3f3f3f3f;
LL s[N];
int n, m;

// 可以通过对 差分数组 单点修改 + 区间查询
// 间接完成对 原数组区 间修改 + 单点查询

// a[i] += x
void modify(int p, int x) {
    for (int i = p; i <= n; i += lowbit(i)) s[i] += x;
}

// return sum[1, x]
LL query(int x) {
    LL ret = 0;
    for (int i = x; i; i -= lowbit(i)) ret += s[i];
    return ret;
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) {
        int x; cin >> x;
        modify(i, x);
    }
    while (m--) {
        int op, x, y; cin >> op >> x >> y;
        // a[x] += y
        if (op == 1) modify(x, y);
        // cout << sum[x, y] << endl;
        else cout << query(y) - query(x - 1) << endl;
    }

    return 0;
}
```

线段树

线段树

```cpp
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1
const int N = 1e5 + 10;


int n, m;
LL a[N];

struct {
        LL l, r, sum, add;
}tr[N << 2];

// 根据左右孩子的sum，刷新父亲的sum
void pushup(int p) {
        tr[p].sum = tr[lc].sum + tr[rc].sum;
}

// p 接受父亲的懒信息 k
void lazy(int p, LL k) {
        auto& [l, r, sum, add] = tr[p];
        sum += (r - l + 1) * k;
        add += k;
}

// 下放 p 的懒信息
void pushdown(int p) {
        lazy(lc, tr[p].add);
        lazy(rc, tr[p].add);

        tr[p].add = 0;
}

void build(int p, int l, int r) {
         tr[p] = {l, r, a[l], 0};
```

```
        if (l == r) return;

        int m = (l + r) >> 1;

        build(lc, l, m); build(rc, m + 1, r);
        pushup(p);
}

// 将[x, y] 区间内每个点染色
// 查找 p 区间染色点的sum
LL query(int p, int x, int y) {
        auto& [l, r, sum, add] = tr[p];

        if (x <= l && r <= y) return sum; // p 中所有点都被染色了

        int m = (l + r) >> 1;
        LL ret = 0;

        pushdown(p);
        if (x <= m) ret += query(lc, x, y); // 处理 p 左区间的染色点的sum
        if (y > m) ret += query(rc, x, y); // 处理 p 区间染色点的sum

        return ret;
}

// [x, y] += k
void modify(int p, int x, int y, LL k) {
        auto& [l, r, sum, add] = tr[p];
        if (x <= l && r <= y) lazy(p, k);
        else {
                int m = (l + r) >> 1;

                pushdown(p);
                if (x <= m) modify(lc, x, y, k);
                if (y > m) modify(rc, x, y, k);
                pushup(p);
        }
}

int main() {

        return 0;

}
```

加 + 乘

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1

const int N = 1e5 + 10;
int n, q, MOD;
LL a[N];

struct {
        int l, r;
        LL sum, k1, b1;
}tr[N << 2];

void pushup(int p) {
        tr[p].sum = (tr[lc].sum + tr[rc].sum) % MOD;
}

void build(int p, int l, int r) {
        tr[p] = {l, r, a[l], 1};
        if (l == r) return;
        int m = (l + r) >> 1;
        build(lc, l, m); build(rc, m + 1, r);
        pushup(p);
}

// 对 p 子树造成 t = k * t + b
void lazy(int p, LL k2, LL b2) {
        auto& [l, r, sum, k1, b1] = tr[p];
        sum = (sum * k2 % MOD + b2 * (r - l + 1)) % MOD;
        b1 = (b1 * k2 % MOD + b2) % MOD; k1 = k1 * k2 % MOD;
}

void pushdown(int p) {
        auto& [l, r, sum, k1, b1] = tr[p];
        lazy(lc, k1, b1); lazy(rc, k1, b1);
        k1 = 1; b1 = 0;
}
```

```
LL query(int p, int x, int y) {
        auto& [l, r, sum, k1, b1] = tr[p];
        if (x <= l && r <= y) return sum;
        LL ret = 0;
        pushdown(p);
        int m = (l + r) >> 1;
        if (x <= m) ret = (ret + query(lc, x, y)) % MOD;
        if (y > m) ret = (ret + query(rc, x, y)) % MOD;
        return ret;
}

void modify(int p, int x, int y, LL k2, LL b2) {
        auto& [l, r, sum, k1, b1] = tr[p];
        if (x <= l && r <= y) lazy(p, k2, b2);
        else {
                pushdown(p);
                int m = (l + r) >> 1;
                if (x <= m) modify(lc, x, y, k2, b2);
                if (y > m) modify(rc, x, y, k2, b2);
                pushup(p);
        }
}

int main() {
        cin >> n >> q >> MOD;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        build(1, 1, n);
        while (q--) {
                int op, x, y; cin >> op >> x >> y;
                if (op == 3) cout << query(1, x, y) << endl;
                else {
                        LL k; cin >> k;
                        if (op == 1) modify(1, x, y, k, 0);
                        else modify(1, x, y, 1, k);
                }
        }

        return 0;
}
```

修改 + 加

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define endl '\n'
#define lc p << 1
#define rc p << 1 | 1

const int N = 1e6 + 10; LL INF = 1e18;
LL a[N];
int n, m;

struct {
        int l, r;
        LL mx;
        bool st; LL x;
        LL add;
}tr[N << 2];

void pushup(int p) {
        tr[p].mx = max(tr[lc].mx, tr[rc].mx);
}

void build(int p, int l, int r) {
        tr[p] = {l, r, a[l]};
        if (l == r) return;
        int m = l + r >> 1;
        build(lc, l, m); build(rc, m + 1, r);
        pushup(p);
}

void lazy(int p, bool st, LL x, LL add) {
        if (st) tr[p].mx = x, tr[p].add = 0, tr[p].st = true, tr[p].x = x;
        tr[p].mx += add, tr[p].add += add;
}

void pushdown(int p) {
        auto& [l, r, mx, st, x, add] = tr[p];
        lazy(lc, st, x, add); lazy(rc, st, x, add);
        st = x = add = 0;
}
```

```
LL query(int p, int x, int y) {
        auto& [l, r, mx, st, k, add] = tr[p];
        if (x <= l && r <= y) return mx;

        pushdown(p);
        LL ret = -INF;
        int m = l + r >> 1;
        if (x <= m) ret = max(ret, query(lc, x, y));
        if (y > m) ret = max(ret, query(rc, x, y));
        return ret;
}

void modify(int p, int x, int y, bool st, LL k, LL add) {
        auto& [l, r, mx, _, __, ___] = tr[p];
        if (x <= l && r <= y) lazy(p, st, k, add);
        else {
                pushdown(p);
                int m = l + r >> 1;
                if (x <= m) modify(lc, x, y, st, k, add);
                if (y > m) modify(rc, x, y, st, k, add);
                pushup(p);
        }
}

int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);
        cin >> n >> m;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        build(1, 1, n);

        while (m--) {
                int op, l, r; cin >> op >> l >> r;
                if (op == 1) {
                        LL x; cin >> x;
                        modify(1, l, r, true, x, 0);
                }
                else if (op == 2) {
                        LL x; cin >> x;
                        modify(1, l, r, false, 0, x);
                }
                else cout << query(1, l, r) << endl;
        }
```

```
        return 0;
}
```

最大子段和

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1

const int N = 5e5 + 10;


LL a[N], n, m;

struct node {
        int l, r;
        LL sum,
        mx, // 最大子段和
        lmx, // 左端点开始最大子段和
        rmx; // 右端点开始的最大子段和
}tr[N << 2];

// p = l + r
// 用 l 和 r 的信息 -> p 的信息
void pushup(node& p, node& l, node& r) {
        p.mx = max(max(l.mx, r.mx), l.rmx + r.lmx);
        p.lmx = max(l.lmx, l.sum + r.lmx);
        p.rmx = max(r.rmx, r.sum + l.rmx);
        p.sum = l.sum + r.sum;
}

void build(int p, int l, int r) {
        tr[p] = {l, r, a[l], a[l], a[l], a[l]};
        if (l == r) return;
        int m = l + r >> 1;
        build(lc, l, m); build(rc, m + 1, r);
        pushup(tr[p], tr[lc], tr[rc]);
}

// a[x] = s
void modify(int p, int x, LL s) {
        auto& [l, r, sum, mx, lmx, rmx] = tr[p];
        if (l == r) sum = mx = lmx = rmx = s;
        else {
```

```
                int m = l + r >> 1;
                if (x <= m) modify(lc, x, s);
                if (x > m) modify(rc, x, s);
                pushup(tr[p], tr[lc], tr[rc]);
        }
}

// 求 p 中染色区间的 node 信息
node query(int p, int x, int y) {
        auto& [l, r, sum, mx, lmx, rmx] = tr[p];
        int m = l + r >> 1;
        if (x <= l && r <= y) return tr[p];
        else if (y <= m) return query(lc, x, y);
        else if (x > m) return query(rc, x, y);
        else {
                // ln : 左边染色区间的信息
                // rn : 右边染色区间的信息
                // ret : p 中染色区间的信息
                node ret, ln = query(lc, x, y), rn = query(rc, x, y);
                pushup(ret, ln, rn);
                return ret;
        }
}

int main() {
        cin >> n >> m;
        for (int i = 1; i <= n; ++i) cin >> a[i];

        build(1, 1, n);

        while (m--) {
                int k, a, b; cin >> k >> a >> b;
                if (k == 1) {
                        if (a > b) swap(a, b);
                        cout << query(1, a, b).mx << endl;
                }
                else modify(1, a, b);
        }
        return 0;
}
```

分治 + 多个信息

0 l r 把 [l,r] 区间内的所有数全变成 0；
1 l r 把 [l,r] 区间内的所有数全变成 1；
2 l r 把 [l,r] 区间内的所有数全部取反，也就是说把所有的 0 变成 1，把所有的 1 变成 0；
3 l r 询问 [l,r] 区间内总共有多少个 1；
4 l r 询问 [l,r] 区间内最多有多少个连续的 1。
修改考虑多个操作
查询考虑分治

```cpp
#include <bits/stdc++.h>
using namespace std;

#define lc p << 1
#define rc p << 1 | 1

const int N = 1e5 + 10, INF = 0x3f3f3f3f;

int a[N], n, m;

struct node {
    int l, r;
    int s1, m1, l1, r1;
    int s0, m0, l0, r0;
    bool f1, f2, f3;
}tr[N << 2];

void pushup(node& p, node& l, node& r) {
    p.s1 = l.s1 + r.s1;
    p.s0 = l.s0 + r.s0;

    p.m1 = max(max(l.m1, r.m1), l.r1 + r.l1);
    p.m0 = max(max(l.m0, r.m0), l.r0 + r.l0);

    p.l1 = l.s0 ? l.l1 : l.l1 + r.l1;
    p.l0 = l.s1 ? l.l0 : l.l0 + r.l0;

    p.r1 = r.s0 ? r.r1 : r.r1 + l.r1;
    p.r0 = r.s1 ? r.r0 : r.r0 + l.r0;
}
```

```
void build(int p, int l, int r) {
    tr[p] = {l, r,
        a[l], a[l], a[l], a[l],
        a[l] ^ 1, a[l] ^ 1, a[l] ^ 1, a[l] ^ 1};

    if (l == r) return;
    int m = l + r >> 1;
    build(lc, l, m); build(rc, m + 1, r);
    pushup(tr[p], tr[lc], tr[rc]);
}

void lazy(int p, bool f1, bool f2, bool f3) {
    auto& [l, r, s1, m1, l1, r1, s0, m0, l0, r0, _, __, ___] = tr[p];
    if (f1)
        s1 = m1 = l1 = r1 = 0,
        s0 = m0 = l0 = r0 = r - l + 1,
        tr[p].f1 = true, tr[p].f2 = tr[p].f3 = false;

    if (f2)
        s1 = m1 = l1 = r1 = r - l + 1,
        s0 = m0 = l0 = r0 = 0,
        tr[p].f2 = true, tr[p].f1 = tr[p].f3 = false;

    if (f3)
        swap(s1, s0), swap(m1, m0), swap(l1, l0), swap(r1, r0),
        tr[p].f3 ^= 1;
}

void pushdown(int p) {
    auto& f1 = tr[p].f1, &f2 = tr[p].f2, &f3 = tr[p].f3;
    lazy(lc, f1, f2, f3); lazy(rc, f1, f2, f3);
    f1 = f2 = f3 = false;
}

void modify(int p, int x, int y, bool f1, bool f2, bool f3) {
    auto& l = tr[p].l, r = tr[p].r;
    if (x <= l && r <= y) lazy(p, f1, f2, f3);
    else {
        pushdown(p);
        int m = l + r >> 1;
        if (x <= m) modify(lc, x, y, f1, f2, f3);
        if (y > m) modify(rc, x, y, f1, f2, f3);
```

```
        pushup(tr[p], tr[lc], tr[rc]);
    }
}

node query(int p, int x, int y) {
    auto& l = tr[p].l, r = tr[p].r;
    if (x <= l && r <= y) return tr[p];
    else {
        pushdown(p);
        int m = l + r >> 1;

        if (y <= m) return query(lc, x, y);
        if (x > m) return query(rc, x, y);

        node ret, nl, nr;

        nl = query(lc, x, y), nr = query(rc, x, y);
        pushup(ret, nl, nr);
        return ret;
    }
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);

    while (m--) {
        int op, l, r;
        cin >> op >> l >> r;
        ++l, ++r;

        if (op == 0) modify(1, l, r, true, false, false);
        else if (op == 1) modify(1, l, r, false, true, false);
        else if (op == 2) modify(1, l, r, false, false, true);
        else if (op == 3) cout << query(1, l, r).s1 << endl;
        else cout << query(1, l, r).m1 << endl;
    }
    return 0;
}
```

权值线段树 + 离散化

```cpp
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define endl '\n'
#define lc p << 1
#define rc p << 1 | 1

const int N = 5e5 + 10;


// 排序去重后的数组，辅助离散化
int disc[N], pos;

int a[N], n;

// 记录当前容器中 每一个数据范围（区间）的元素个数
// [l, r] : sum 表示位次在该区间的元素个数
struct {
        int l, r, sum;
}tr[N << 2];

// 找 x 离散化后的值
int id(int x, int l = 1, int r = pos) {
        while (l < r) {
                int m = l + r >> 1;
                if (disc[m] < x) l = m + 1;
                else r = m;
        }
        return l;
}

void pushup(int p) {
        tr[p].sum = tr[lc].sum + tr[rc].sum;
}

void build(int p, int l, int r) {
        tr[p] = {l, r};
        if (l == r) return;
        int m = l + r >> 1;
        build(lc, l, m); build(rc, m + 1, r);
}
```

```
void modify(int p, int x) {
        auto& [l, r, sum] = tr[p];
        if (l == r) ++sum;
        else {
                int m = l + r >> 1;
                if (x <= m) modify(lc, x);
                if (x > m) modify(rc, x);
                pushup(p);
        }
}

int query(int p, int x, int y) {
        auto& [l, r, sum] = tr[p];
        if (x <= l && r <= y) return sum;

        int m = l + r >> 1;
        int ret = 0;
        if (x <= m) ret = ret + query(lc, x, y);
        if (y > m) ret = ret + query(rc, x, y);
        return ret;
}

int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);

        cin >> n;
        for (int i = 1; i <= n; ++i) cin >> a[i];

        // 离散化
        for (int i = 1; i <= n; ++i) disc[++pos] = a[i];
        sort(disc + 1, disc + n + 1);
        pos = unique(disc + 1, disc + n + 1) - (disc + 1);

        build(1, 1, pos); // 注意线段树维护【1, pos】

        LL ret = 0;
        for (int i = 1; i <= n; ++i) {
                int x = id(a[i]);
                ret = ret + query(1, x + 1, pos);
                modify(1, x);
        }
        cout << ret << endl;
```

```
        return 0;
}
```

区间开更号（剪枝）

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1
const int N = 1e5 + 10;


LL a[N];
int n, m;

struct {
        int l, r;
        LL sum, mx; // 区间最大值用于剪枝
}tr[N << 2];

void pushup(int p) {
        tr[p].sum = tr[lc].sum + tr[rc].sum;
        tr[p].mx = max(tr[lc].mx, tr[rc].mx);
}

void build(int p, int l, int r) {
        tr[p] = {l, r, a[l], a[l]};
        if (l == r) return;
        int m = l + r >> 1;
        build(lc, l, m); build(rc, m + 1, r);
        pushup(p);
}

void modify(int p, int x, int y) {
        auto& [l, r, sum, mx] = tr[p];
        if (mx == 1) return; // 该区间最大值是 1, 不必再开更号, 撤
        if (l == r) sum = mx = sqrt(sum);
        else {
                int m = l + r >> 1;
                if (x <= m) modify(lc, x, y);
                if (y > m) modify(rc, x, y);
                pushup(p);
        }
}
```

```
LL query(int p, int x, int y) {
        auto& [l, r, sum, mx] = tr[p];
        if (x <= l && r <= y) return sum;

        LL ret = 0;
        int m = l + r >> 1;
        if (x <= m) ret += query(lc, x, y);
        if (y > m) ret += query(rc, x, y);
        return ret;
}

int main() {
        cin >> n;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        build(1, 1, n);

        cin >> m;
        while (m--) {
                int k, l, r; cin >> k >> l >> r;
                if (l > r) swap(l, r);
                if (k) cout << query(1, l, r) << endl;
                else modify(1, l, r);
        }

        return 0;
}
```

方差

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1

const int N = 1e5 + 10, INF = 0x3f3f3f3f, MOD = 1e9 + 7;

LL a[N], n, m;

struct node {
    int l, r;
    LL sum, sum2; // 维护区间sum和，区间平方和，  间接维护方差信息
}tr[N << 2];

// 用 l 和 r 拼接 p
void pushup(node& p, node& l, node& r) {
    p.sum = (l.sum + r.sum) % MOD;
    p.sum2 = (l.sum2 + r.sum2) % MOD;
}

void build(int p, int l, int r) {
    tr[p] = {l, r, a[l], a[l] * a[l] % MOD};
    if (l == r) return;
    int m = l + r >> 1;
    build(lc, l, m); build(rc, m + 1, r);
    pushup(tr[p], tr[lc], tr[rc]);
}

node query(int p, int x, int y) {
    auto& [l, r, sum, sum2] = tr[p];
    if (x <= l && r <= y) return tr[p];

    int m = l + r >> 1;
    if (y <= m) return query(lc, x, y);
    else if (x > m) return query(rc, x, y);
    else {
        node ret, ln, rn;
        ln = query(lc, x, y); rn = query(rc, x, y);
        pushup(ret, ln, rn);
```

```
        return ret;
    }
}

void modify(int p, int x, LL k) {
    auto& [l, r, sum, sum2] = tr[p];
    if (l == r) {
        sum = k; sum2 = k * k % MOD;
        return;
    }

    int m = l + r >> 1;
    if (x <= m) modify(lc, x, k);
    else modify(rc, x, k);

    pushup(tr[p], tr[lc], tr[rc]);
}

LL qpow(LL a, LL b) {
    LL ret = 1;
    while (b) {
        if (b & 1) ret = ret * a % MOD;
        a = a * a % MOD, b >>= 1;
    }
    return ret;
}

int main() {
    cin >> n >> m;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    build(1, 1, n);

    while (m--) {
        LL op, x, y; cin >> op >> x >> y;
        if (op == 1) modify(1, x, y);
        else {
            auto[_, __, sum, sum2] = query(1, x, y);
            LL len = y - x + 1;
            LL k = qpow(len, MOD - 2);
            cout << ((sum2 * k % MOD - (sum * k % MOD) * (sum * k % MOD) % MOD) % MOD + MOD) % M
        }
    }
```

```
    return 0;
}
```

GCD

```cpp
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define lc p << 1
#define rc p << 1 | 1

const int N = 5e5 + 10, INF = 0x3f3f3f3f;

LL a[N], n, q;

struct {
    int l, r;
    LL sum, d; // 维护差分数组的 sum 和 gcd
}tr[N << 2];

void pushup(int p) {
    tr[p].sum = tr[lc].sum + tr[rc].sum;
    tr[p].d = gcd(tr[lc].d, tr[rc].d);
}

void build(int p, int l, int r) {
    tr[p] = {l, r, a[l], a[l]};
    if (l == r) return;

    int m = l + r >> 1;
    build(lc, l, m); build(rc, m + 1, r);
    pushup(p);
}

// 维护的是差分数组，区间修改变成了单点修改
void modify(int p, int x, LL k) {
    auto& [l, r, sum, d] = tr[p];
    if (l == r) {
        sum += k; d += k;
        return;
    }

    int m = l + r >> 1;
    if (x <= m) modify(lc, x, k);
    else modify(rc, x, k);
```

```
    pushup(p);
}

LL query_sum(int p, int x, int y) {
    auto& [l, r, sum, d] = tr[p];
    if (x <= l && r <= y) return sum;

    LL ret = 0;
    int m = l + r >> 1;
    if (x <= m) ret += query_sum(lc, x, y);
    if (y > m) ret += query_sum(rc, x, y);

    return ret;
}

LL query_d(int p, int x, int y) {
    auto& [l, r, sum, d] = tr[p];
    if (x <= l && r <= y) return d;

    LL ret = 0;
    int m = l + r >> 1;
    if (x <= m) ret = gcd(ret, query_d(lc, x, y));
    if (y > m) ret = gcd(ret, query_d(rc, x, y));

    return ret;
}

// 数组的 gcd 和它对应的 差分数组 gcd 一致
// gcd(a, b, c, d) == gcd(a, b - a, c - b, d - c)
// 本题是求右边，从而间接求出左边

int main() {
    cin >> n >> q;
    for (int i = 1; i <= n; ++i) {
        LL x; cin >> x;
        a[i] += x; a[i + 1] -= x;
    }

    build(1, 1, n);

    while (q--) {
        char op; int x, y;
        cin >> op >> x >> y;
```

```
        if (op == 'Q') {
            // 根据公式：第一项为原数组的值，用 sum[1, x] 表示
            LL sum = query_sum(1, 1, x);
            LL d = 0;
            // 根据公式：其余项为差分数组值，直接求 gcd
            if (x + 1 <= y) d = query_d(1, x + 1, y);

            // ret 为所有项的 gcd
            LL ret = gcd(sum, d);

            // 由于存在减法操作，可能出现负数，此处直接补正即可
            cout << abs(ret) << endl;
        }
        else {
            LL k; cin >> k;
            modify(1, x, k);
            if (y + 1 <= n) modify(1, y + 1, -k);
        }
    }

    return 0;
}
```

ST表

尾插操作

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

const int N = 2e5 + 10;
const LL INF = 1e18;

LL mx[N][25];

LL query(int l, int r) {
    int k = log2(r - l + 1);
    return max(mx[l][k], mx[r - (1 << k) + 1][k]);
}

// 黑盒：ST表维护的是[1, n] 所有子区间的信息

// x 是尾插后的最后下表
void modify(int x, LL k) {
    // 刷新尾插所造成影响的区间
    mx[x][0] = k;
    for (int j = 1; x - (1 << j) + 1 >= 1; ++j) {
        int i = x - (1 << j) + 1;
        mx[i][j] = max(query(i, x - 1), k);
    }
}

int main() {
    int q, d;
    cin >> q >> d;
    LL end = 0, pre = 0;

    while (q--) {

        char op; LL x;
        cin >> op >> x;
        if (op == 'A') modify(++end, (pre + x) % d);
        else cout << (pre = query(end - x + 1, end)) << endl;
    }
```

```
    return 0;
}
```

ST表

```cpp
#include <bits/stdc++.h>
using namespace std;
#define endl '\n'

const int N = 1e5 + 10;

int a[N], n, m;
int mx[N][20]; // mx[i][j] : max[i, i + 2 ^ j)
int lg[N];

// 初始化 ST表
void init() {
        lg[0] = -1;
        for (int i = 1; i <= n; ++i) {
                lg[i] = lg[i >> 1] + 1;
                mx[i][0] = a[i];
        }

        // 用 短区间 更新 长区间，j从小到大
        for (int j = 1; j <= lg[n]; ++j)
                for (int i = 1; i + (1 << j) - 1 <= n; ++i)
                        mx[i][j] = max(mx[i][j - 1], mx[i + (1 << j - 1)][j - 1]);
}

int query(int l, int r) {
        int k = lg[r - l + 1];
        return max(mx[l][k], mx[r - (1 << k) + 1][k]);
}

int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);
        cin >> n >> m;
        for (int i = 1; i <= n; ++i) cin >> a[i];
        init();
        for (int i = 1; i <= m; ++i) {
                int l, r; cin >> l >> r;
                cout << query(l, r) << endl;
        }

        return 0;
        }
```

第2章 动态规划

树形dp

基本树形dp

最大独立集

```
#include <bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10, INF = 0x3f3f3f3f;
int dp1[N], dp0[N], n;

vector<int> e[N];

void dfs(int u, int fa) {
    for (auto v : e[u]) {
        if (fa == v) continue;
        dfs(v, u);

        if (dp0[v] > 0) dp1[u] += dp0[v];

        int x = max(dp0[v], dp1[v]);
        if (x > 0) dp0[u] += x;
    }
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; ++i) cin >> dp1[i];

    for (int i = 1; i < n; ++i) {
        int u, v; cin >> u >> v;
        e[v].emplace_back(u);
        e[u].emplace_back(v);
    }

    dfs(1, 1);

    cout << max(dp1[1], dp0[1]) << endl;
```

```
    return 0;
}
```

最小边覆盖

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 2e5 + 10, INF = 0x3f3f3f3f;
int f[N]; // 必选
int g[N]; // 必不选

vector<int> e[N];

void dfs(int x, int p) {
    for (auto y : e[x]) {
        if (y == p) continue;
        dfs(y, x);
        f[x] += min(f[y], g[y]);
        g[x] += f[y];
    }
}

int main() {
    int n; cin >> n;
    for (int i = 0; i < n; ++i) f[i] = 1;

    for (int i = 1; i <= n; ++i) {
        int u; cin >> u;
        int k; cin >> k;
        while (k--) {
            int v; cin >> v;
            e[u].emplace_back(v); e[v].emplace_back(u);
        }
    }

    dfs(0, 0);
    cout << min(f[0], g[0]) << endl;

    return 0;
}
```

最小点覆盖

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e4 + 10, INF = 0x3f3f3f3f;
int n;
vector<int> e[N];

int f[N], g[N], h[N];

void dfs(int x, int p) {
    int mn = INF;
    for (auto y : e[x]) {
        if (y == p) continue;
        dfs(y, x);
        h[x] += min(f[y], g[y]);
        f[x] += min(f[y], min(g[y], h[y]));
        g[x] += min(f[y], g[y]);
        mn = min(f[y] - g[y], mn);
    }
    if (mn > 0) g[x] += mn;
}

int main() {
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int x, y; cin >> x >> y;
        e[x].emplace_back(y); e[y].emplace_back(x);
    }
    for (int i = 1; i <= n; ++i) f[i] = 1;
    dfs(1, 1);
    cout << min(f[1], g[1]);

    return 0;
}
```

重心与直径

重心

```
<p>
 定义:
 在一棵树上有 n 个 分子，每一个分子上有若干个节点，节点总数为 m
 g[i] : 将第 i 个 分子 及其 相连的边删去，生成的若干子树中，节点个数最多的 子树的 节点个数
 g[core] = min(g[N])

 性质 1 : 重心最多有两个，如果有两个，那么两重心一定相邻
 性质 2 : 所有子树节点个数不会超过 m / 2 (向下取整)
 性质 3 :
     dist[i] : 所有节点到 i 分子的距离总和
     dist[core] = min(dist[N])
</p>
```

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

const int N = 5e4 + 10, INF = 0x3f3f3f3f;
vector<int> e[N];
int n;

// i分子 为根的 子树的 节点个数
int f[N];

int g[N];

// 返回 以 x分子 为根的子树的节点个数
int dfs1(int x, int fa) {
    f[x] = 1;

    for (auto y : e[x]) {
        if (y == fa) continue;

        int k = dfs1(y, x);

        f[x] += k;
        g[x] = max(g[x], k);
    }
```

```
    g[x] = max(g[x], n - f[x]);
    return f[x];
}

// 所有子节点走到 x 的距离总和
LL dfs2 (int x, int fa) {
    LL ret = 0;
    for (auto y : e[x]) {
        if (y == fa) continue;
        int k = dfs2(y, x);
        // y 的子节点共 f[y] 个
        // 这 f[y] 个节点先走到 y 共走了 k 步
        // 然后他们再一起走一步到 x，共 f[y] 步
        ret += k + f[y];
    }
    return ret;
}

int main() {
    cin >> n;
    for (int i = 1; i < n; ++i) {
        int x, y; cin >> x >> y;
        e[x].push_back(y); e[y].push_back(x);
    }

    // 以 1 为根节点刷新 f, g
    dfs1(1, 0);

    int mn = INF;

    for (int i = 1; i <= n; ++i) mn = min(mn, g[i]);

    int core; // 重心

    for (int i = 1; i <= n; ++i) {
        if (g[i] == mn) {
            core = i;
            break;
        }
    }

    memset(f, 0, sizeof f);
```

```
    dfs1(core, 0); // 用重心为根，刷新 f 数组
    cout << core << ' ';
    cout << dfs2(core, 0) << endl;

    return 0;
}
```

带点权边权重心

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;
using PII = pair<int, int>;

const int N = 1e5 + 10, INF = 0x3f3f3f3f;

vector<PII> e[N];

LL n, m;

// f[i] : i 子树节点个数
// g[i] : 去除 i 后，最大子树节点个数
LL w[N], f[N], g[N];

// 刷新 f, g
LL dfs1(int x, int fa) {
        f[x] = w[x];

        for (auto [y, l] : e[x]) {
                if (y == fa) continue;

                LL k = dfs1(y, x);

                f[x] += k;
                g[x] = max(g[x], k);
        }

        g[x] = max(g[x], m - f[x]);

        return f[x];
}

LL dfs2(int x, int fa) {
        LL ret = 0;

        for (auto [y, l] : e[x]) {
                if (y == fa) continue;

                LL k = dfs2(y, x);
```

```
                ret += k + f[y] * l;
        }
        return ret;
}

int main() {
        cin >> n;
        for (int i = 1; i <= n; ++i) {
                cin >> w[i];
                m += w[i];
        }

        for (int i = 1; i < n; ++i) {
                int x, y, l;
                cin >> x >> y >> l;
                e[x].emplace_back(y, l); e[y].emplace_back(x, l);
        }

        dfs1(1, 0);

        LL mn = INF;
        for (int i = 1; i <= n; ++i) mn = min(mn, g[i]);

        int core;
        for (int i = 1; i <= n; ++i)
                if (g[i] == mn) core = i;

        memset(f, 0, sizeof f);
        dfs1(core, 0);

        cout << dfs2(core, 0) << endl;

        return 0;
}
```

带点权重心

```
#include
<bits/stdc++.h> using
namespace std;

using LL = long long;

const int N = 1e2 + 10, INF =
0x3f3f3f3f; vector<int> e[N];

LL n, m; // 分子个数 / 节点个数

// i 分子的节点个数
int w[N];

// 子树节点个数
LL f[N];

// 去除 i，最多节点的 生成子树的 节点个数
LL g[N];

// 返回 以 x 为根的子树节点个数
LL dfs1(int x, int fa) {
    f[x] = w[x];

    for (auto y : e[x]) {
        if (y == fa) continue;

        LL k = dfs1(y, x);

        f[x] += k;
        g[x] = max(g[x],
    k); }
    g[x] = max(g[x], m - f[x]);
    return
f[x]; }

// 返回 x 所有子节点到 x 的总步数
LL dfs2(int x, int fa) {
    LL ret = 0;
    for (auto y : e[x]) {
        if (y == fa) continue;

        int k = dfs2(y, x);
```

```
        ret += k + f[y];
    }
    return ret;
}

int main() {
    cin >> n;
    for (int i = 1; i <= n; ++i) {
        cin >> w[i];
        m += w[i];

        // 左链接 右链接
        int l, r; cin >> l >> r;

        if (l) e[i].push_back(l), e[l].push_back(i);
        if (r) e[i].push_back(r), e[r].push_back(i);
    }

    // 以 1 为根节点刷新 f, g
    dfs1(1, 0);

    LL mn = INF;
    for (int i = 1; i <= n; ++i) mn = min(mn, g[i]);

    int core;
    for (int i = 1; i <= n; ++i)
        if (g[i] == mn) core = i;

    //  以 core 为根节点刷新 f
    memset(f, 0, sizeof f);
    dfs1(core, 0);

    cout << dfs2(core, 0) << endl;

    return 0;
}
```

LCA

倍增法

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'

const int N = 5e5 + 10;
vector<int> e[N];
int f[N][25]; // f[x][i] : x 向上跳 2 ^ i 步的节点编号
int dep[N]; // dep[x] : x 所在的层数

void dfs(int x, int fa) {
        dep[x] = dep[fa] + 1;

        f[x][0] = fa;

        for (int i = 1; i <= 20; ++i)
                f[x][i] = f[f[x][i - 1]][i - 1];

        for (auto y : e[x]) {
                if (y == fa) continue;

                dfs(y, x);
        }
}

int lca(int x, int y) {
        if (dep[x] < dep[y]) swap(x, y);

        // x 跳到与 y 同层
        for (int i = 20; i >= 0; --i)
                if (dep[f[x][i]] >= dep[y]) x = f[x][i];

        // y 是 x 的祖先，x 跳到 dep[y]时，刚好到 y
        if (x == y) return y;

        // x 与 y 同时向上跳，跳到 lca(x, y) 的下面
        for (int i = 20; i >= 0; --i) {
                if (f[x][i] != f[y][i])
                        x = f[x][i], y = f[y][i];
```

```
        }
        return f[x][0];
}

int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);

        // n 个节点，m 次查询，s 为根节点
        int n, m, s;
        cin >> n >> m >> s;

        for (int i = 1; i < n; ++i) {
                int x, y;
                cin >> x >> y;
                e[x].push_back(y); e[y].push_back(x);
        }

        dfs(s, 0);

        for (int i = 1; i <= m; ++i) {
                int x, y;
                cin >> x >> y;
                cout << lca(x, y) << endl;
        }

        return 0;
}
```

Tarjan

```cpp
#include <bits/stdc++.h>
using namespace std;
using PII = pair<int, int>;

const int N = 5e5 + 10;
vector<int> e[N];
int fa[N]; // 并查集
int st[N]; // 是否遍历过

// q[x] : x 有关查询的所有的另一半 y, 以及问题的编号id
vector<PII> q[N];
int ret[N];

int find(int x) {
        return x == fa[x] ? x : fa[x] = find(fa[x]);
}

void tarjan(int x) {
        st[x] = true;
        for (auto y : e[x]) {
                if (st[y]) continue;

                tarjan(y);
                fa[y] = x; // 回溯时合并，子树向根合并
        }

        for (auto [y, id] : q[x]) {
                if (st[y] == false) continue;
                ret[id] = find(y); // y 已经遍历，y 的根是 y 的代表元
        }
}

int main() {
        int n, m, s;
        cin >> n >> m >> s;

        for (int i = 1; i <= n; ++i) fa[i] = i;

        for (int i = 1; i < n; ++i) {
                int x, y; cin >> x >> y;
                e[x].emplace_back(y); e[y].emplace_back(x);
```

```
        }

        for (int i = 1; i <= m; ++i) {
                int x, y; cin >> x >> y;
                q[x].emplace_back(y, i); q[y].emplace_back(x, i);
        }

        tarjan(s);

        for (int i = 1; i <= m; ++i) cout << ret[i] << endl;

        return 0;
}
```

树上两点距离

```cpp
#include <bits/stdc++.h>
using namespace std;
using PII = pair<int, int>;
const int N = 2e5 + 10, INF = 0x3f3f3f3f;

vector<PII> e[N];
int f[N][25]; // f[x][i] : x 向上跳 2 ^ i 步的节点编号
int dep[N]; // dep[x] : x 所在的层数
int dist[N]; // 根到每一个节点的距离

void dfs(int x, int fa) {
        dep[x] = dep[fa] + 1;

        f[x][0] = fa;

        for (int i = 1; i <= 20; ++i)
                f[x][i] = f[f[x][i - 1]][i - 1];

        for (auto [y, w] : e[x]) {
                if (y == fa) continue;

        dist[y] = dist[x] + w;
                dfs(y, x);
        }
}

int lca(int x, int y) {
        if (dep[x] < dep[y]) swap(x, y);

        // x 跳到与 y 同层
        for (int i = 20; i >= 0; --i)
                if (dep[f[x][i]] >= dep[y]) x = f[x][i];

        // y 是 x 的祖先，x 跳到 dep[y]时，刚好到 y
        if (x == y) return y;

        // x 与 y 同时向上跳，跳到 lca(x, y) 的下面
        for (int i = 20; i >= 0; --i) {
                if (f[x][i] != f[y][i])
                        x = f[x][i], y = f[y][i];
        }
```

```
        return f[x][0];
}

int main() {
    int n, m; cin >> n >> m;

    for (int i = 1; i < n; ++i) {
        int x, y, w; cin >> x >> y >> w;
        e[x].emplace_back(y, w); e[y].emplace_back(x, w);
    }

    //  谁是根都不重要
    dfs(1, 0);

    for (int i = 1; i <= m; ++i) {
        int x, y; cin >> x >> y;
        cout << dist[x] + dist[y] - 2 * dist[lca(x, y)] << endl;
    }

    return 0;
}
```

树上询问

```
<p>
  查询 有多少个节点作为根节点时，lca(a, b) == c
  本质：一般情况下，以 c 为核心，除了a, b所在分支，其余节点均可
</p>
```

```
#include <bits/stdc++.h>
using namespace std;
const int N = 5e5 + 10, INF = 0x3f3f3f3f;
vector<int> e[N];
int dep[N];
int f[N][25];
int sum[N]; //  以该节点为根的子树的 节点个数

void dfs(int x, int fa) {
    dep[x] = dep[fa] + 1;
    sum[x] = 1;

    f[x][0] = fa;
    for (int i = 1; i <= 20; ++i)
        f[x][i] = f[f[x][i - 1]][i - 1];

    for (auto y : e[x]) {
        if (y == fa) continue;
        dfs(y, x);
        sum[x] += sum[y];
    }
}

// x 向上跳，跳到 depth 那一层
int up(int x, int depth) {
    for (int i = 20; i >= 0; --i)
        if (dep[f[x][i]] >= depth) x = f[x][i];
    return x;
}

int lca(int x, int y) {
    if (dep[x] < dep[y]) swap(x, y);
    x = up(x, dep[y]);

    if (x == y) return y;
```

```
    for (int i = 20; i >= 0; --i)
        if (f[x][i] != f[y][i]) x = f[x][i], y = f[y][i];

    return f[x][0];
}

int main() {
    int n, m;
    cin >> n >> m;
    for (int i = 1; i < n; ++i) {
        int x, y; cin >> x >> y;
        e[x].push_back(y); e[y].push_back(x);
    }

    dfs(1, 0);

    for (int i = 1; i <= m; ++i) {
        int a, b, c;
        cin >> a >> b >> c;

        // 三点重合，所有点都行
        if (a == b && b == c) cout << n << endl;
        // 考虑两两重合

        // a 和 b 重合，必定无解
        else if (a == b) cout << 0 << endl;
        // a 和 c 重合
        else if (a == c) {
            // 除了 b 所在的分支都行
            if (lca(b, c) == c) {
                b = up(b, dep[c] + 1);
                cout << n - sum[b] << endl;
            }
            else cout << sum[c] << endl;
        }
        else if (b == c) {
            // 除了 a 所在的分支都行
            if (lca(a, c) == c) {
                a = up(a, dep[c] + 1);
                cout << n - sum[a] << endl;
            }
            else cout << sum[c] << endl;
```

```
        }
        // 考虑没有重合的情况
        // 若有解，则a, b都是 c 的孩子，或者一个是 c 的孩子，而另一个不是

        // 先考虑 a, b 都是 c 的孩子
        else if (lca(a, c) == c && lca(b, c) == c) {
            // a 和 b 在 c 的不同分枝
            if (lca(a, b) == c) {
                a = up(a, dep[c] + 1);
                b = up(b, dep[c] + 1);
                cout << n - sum[a] - sum[b] << endl;
            }
            // a 和 b 在 c 的同一个分枝
            else cout << 0 << endl;
        }
        // 再考虑一个是 c 的孩子，而另一个不是

        // a 是 b 不是
        else if (lca(b, c) == c && lca(a, c) != c) {
            b = up(b, dep[c] + 1);
            cout << sum[c] - sum[b] << endl;
        }
        // b 是 a 不是
        else if(lca(a, c) == c && lca(b, c) != c) {
            a = up(a, dep[c] + 1);
            cout << sum[c] - sum[a] << endl;
        }
        // 其余情况均无解
        else cout << 0 << endl;
    }

    return 0;
}
```

数位dp

萌数

蒟蒻 SOL 居然觉得数很萌！
好在在他眼里，并不是所有数都是萌的。只有满足“存在长度至少为 2 的回文子串”的数是萌的——
也就是说，101 是萌的，因为 101 本身就是一个回文数；110 是萌的，因为包含回文子串 11；
但是 102 不是萌的，1201 也不是萌的。
现在 SOL 想知道从 l 到 r 的所有整数中有多少个萌数。
由于答案可能很大，所以只需要输出答案对 1000000007（1e9+7）的余数。

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;
const int N = 1e3 + 10, INF = 0x3f3f3f3f, MOD = 1e9 + 7;

LL f[N][10][10][2][2][2];
int a[N], n;

// p1 p2 p
// f1 表示 p1 为非前导0合法位
// f2 表示 p2 为非前导0合法位
// ff 是萌数，数字合法
LL dfs(int p, int p1, int p2, bool f1, bool f2, bool ff, bool limit, bool zero) {
    if (p == 0) return ff;
    if (!limit && ~f[p][p1][p2][f1][f2][ff]) return f[p][p1][p2][f1][f2][ff];

    LL ret = 0;
    int up = limit ? a[p] : 9;

    for (int i = 0; i <= up; ++i) {
        ret = (ret + dfs(p - 1, p2, i, f2, !(zero && i == 0),
        ff || i == p1 && f1 || i == p2 && f2, limit && i == a[p], zero && i == 0)) % MOD;
    }

    if (!limit) return f[p][p1][p2][f1][f2][ff] = ret;
    else return ret;
}

LL calc(string x) {
```

```
    if (x.size() <= 1) return 0;

    memset(f, -1, sizeof f);
    n = x.size(); x = ' ' + x;
    for (int i = 1; i <= n; ++i) a[i] = x[n - i + 1] - '0';

    return dfs(n, 0, 0, false, false, false, true, true);
}

bool cheak(string s) {
    int n = s.size();
    s = ' ' + s;
    if (s[2] == s[1]) return true;
    for (int i = 3; i <= n; ++i)
        if (s[i] == s[i - 1] || s[i] == s[i - 2]) return true;
    return false;
}

int main() {
    string x, y; cin >> x >> y;
    LL ret = ((calc(y) - calc(x)) % MOD + MOD) % MOD;
    if (cheak(x)) ret = (ret + 1) % MOD;
    cout << ret;
    return 0;
}
```

数位dp(枚举区间不降数)

```cpp
#include <bits/stdc++.h>
using namespace std;
const int N = 15, INF = 0x3f3f3f3f;

int a[N], n;

int f[N][N]; //

// 从 p 位置开始填，前一位是 prev，限制为limit，共有多少种
// limit : 父亲都是上届
int dfs(int p, int prev, bool limit) {
    if (p == 0) return 1;
    if (~f[p][prev][limit]) return f[p][prev][limit];

    int end = 9;
    if (limit) end = a[p]; // 父亲都是上界

    int ret = 0;

    for (int i = prev; i <= end; ++i)
        ret += dfs(p - 1, i, limit && (i == a[p]));
    // 当所有父亲都是上界，我也是上界，那么我的孩子的所有父亲都是上届

    return f[p][prev][limit] = ret;
}

// 统计[0, x]，不降数的个数
int calc(int x) {
    if (x == 0) return 1;

    memset(f, -1, sizeof f);
    n = 0;

    while (x) {
        a[++n] = x % 10;
        x /= 10;
    }

    return dfs(n, 0, true);
}
```

```
int main() {
    int a, b;
    while (cin >> a >> b) {
        cout << calc(b) - calc(a - 1) << endl;
    }

    return 0;
}
```

状压dp

棋盘覆盖3

```
<p>
  在 n * n * n 的正方体中选 n 个 数，使得sum最小
  每个（ x, y, z）平面只选一个
</p>


#include <bits/stdc++.h>
using namespace std;
using LL = long long;

#define endl '\n'

const int N = 12, INF = 0x3f3f3f3f;
int a[N + 1][N][N], n;

// fst[i]: 第 i 个 z 平面的 f1和f2, 有且仅有 i 个1，筛选合法状态
vector<int> fst[N + 1];

// f[i][f1][f2], 在 【0，i】 号 z 平面挑选，
//0 不挑，x 平面挑选了 f1， y平面挑了 f2，最小sum

// ret = f[n][1111111][1111111]

int f[N + 1][1 << N][1 << N];

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);

    cin >> n;
    for (int k = 1; k <= n; ++k)
        for (int i = 0; i < n; ++i)
            for (int j = 0; j < n; ++j)
                cin >> a[k][i][j];

    for (int st = 0; st < (1 << n); ++st) {
        int sum = 0; // st 的 1 的个数
        for (int i = 0; i < n; ++i)
            if ((st >> i) & 1) ++sum;
```

```
        fst[sum].emplace_back(st);
    }

    memset(f, 0x3f, sizeof f);
    f[0][0][0] = 0;

    for (int k = 1; k <= n; ++k)
        for (auto x : fst[k])
            for (auto y : fst[k])
                // 枚举第 k 个 z 平面可能选的点
                for (int i = 0; i < n; ++i)
                    if ((x >> i) & 1)
                        for (int j = 0; j < n; ++j)
                            if ((y >> j) & 1)
                                f[k][x][y] = min(f[k][x][y],
                                        a[k][i][j] + f[k - 1][x ^ (1 << i)][y ^ (1 << j)]);

    cout << f[n][(1 << n) - 1][(1 << n) - 1] << endl;

    return 0;
}
```

TSP

```
<p>
  以 0 为起点，n - 1 为终点，最小的排列
</p>


#include <bits/stdc++.h>
using namespace std;
const int N = 20, INF = 0x3f3f3f3f;
int n;
int e[N][N];

// f[st][i] : 由 st 组成，以 0为起点， i 为结尾，的最小排列
int f[1 << N][N];

int main() {
    cin >> n;
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            cin >> e[i][j];

    memset(f, 0x3f, sizeof f);
    f[1][0] = 0; // st仅包含 0 时，当终点为 0 时有效，排列权值为 0

    for (int st = 2; st < (1 << n); ++st)
        if (st & 1) // 以 0 为起点，0 必须包含
            for (int i = 0; i < n; ++i) // 落点
                if ((st >> i) & 1)
                    for (int j = 0; j < n; ++j) // 前一步的落点
                        if (((st >> j) & 1) && j != i)
                            f[st][i] = min(f[st][i], f[st ^ (1 << i)][j] + e[j][i]);

    cout << f[(1 << n) - 1][n - 1];

    return 0;
}
```

第3章 图论

01bfs

```
void _01bfs() {
    vector<int> dist(n + 10, INF);
    dist[1] = 0;

    deque<int> q;
    q.emplace_back(1);

    vector<bool> st(n + 10, false);

    while (q.size()) {
        auto x = q.front(); q.pop_front();

        if (st[x]) continue;
        st[x] = true;

        for (auto [y, w] : e[x]) {
            if (dist[x] + w < dist[y]) {
                dist[y] = dist[x] + w;
                if (w == 0) q.push_front(y);
                else q.push_back(y);
            }
        }
    }
}
```

dijkstra

```
<p>
    用【1, m】的边建图，从 1->n, 最短路径
    s 为起点，d 为终点（ m预处理用的）
</p>


int dijkstra(int m, int s, int d) {
    init(m); // 建图
    memset(dist, 0x3f, sizeof dist);
    dist[s] = 0;

    // 注意：默认大根堆
    // first: w
    // second: x(节点)
    priority_queue<PII, vector<PII>, greater<PII>> q; q.emplace(0, s);
    while (q.size()) {
        auto [w, x] = q.top(); q.pop();
        if (x == d) return dist[d]; // 终点(x)已更新，撤

        // 防止已更新节点重复松弛
        // st 表示节点是否完成更新
        if (st[x]) continue;
        st[x] = true;

        // x -> y, 用 x 松弛 y
        for (auto [y, w] : e[x]) {
            if (dist[x] + w < dist[y]) {
                dist[y] = dist[x] + w;
                q.emplace(dist[y], y);
            }
        }
    }
    return INF; // 所有可达点都松弛，但没见到终点，不连通，默认无穷大
}
```

第4章 数学知识

数论

分解质因子

```
void deprime(int x) {
    for (int i = 2; i <= x / i; ++i) {
        if (x % i == 0) {
            p[++pos] = i;
            while (x % i == 0) a[pos]++, x /= i;
        }
    }
    if (x != 1) p[++pos] = x, a[pos] = 1;
}
```

EXGCD

```
<p>
  返回值：gcd(a, b)
  ax + by = gcd(a, b)，将 x, y 赋值成满足等式的特解
  特殊判断：gcd(a, 0) = a
</p>


LL exgcd(LL a, LL b, LL& x, LL& y) {
    if (b == 0) {
        x = 1;
        y = 0;
        return a;
    }

    LL x1, y1, d;
    d = exgcd(b, a % b, x1, y1);

    x = y1;
    y = x1 - a / b * y1;
    return d;
}

// 求 a 在 MOD 意义下的乘法逆元
int re(int a) {
    int x, y;
    exgcd(a, MOD, x, y);
    return (x % MOD + MOD) % MOD;
}

void func() {
    LL a, b, x, y;
    LL d = exgcd(a, b, x, y);

    // 小写表示特解，大写表示通解
    // k 是任意整数
    X = x + k * (b / d)
    Y = y - k * (a / d)
}
```

线性代数

矩阵快速幂

```
<p>
  做斐波那契时，注意前面几项需要特殊判断，否则会TLE(死循环)
</p>



#include <bits/stdc++.h>
using namespace std;
using LL = long long;
const int N = 110, MOD = 1e9 + 7;

// n 是矩阵大小，用矩阵快速幂时，往往需要手动赋值
int n;

// 封装矩阵，实现重载乘法
struct mat {
        int m[N][N];
        mat() {
                memset(m, 0, sizeof m);
        }
        mat operator*(const mat& A)const {
                mat ret;
                for (int i = 1; i <= n; ++i)
                        for (int j = 1; j <= n; ++j)
                                for (int k = 1; k <= n; ++k)
                                        ret.m[i][j] = (ret.m[i][j] + m[i][k] * A.m[k][j]) % MOD
                return ret;
        }
};

// ret = a ^ k
mat qpow(mat a, LL b) {
        mat ret;
        for (int i = 1; i <= n; ++i) ret.m[i][i] = 1; // 初始化成单位矩阵，方便乘法运算

        while (b) {
                if (b & 1) ret = ret * a;
                b >>= 1; a = a * a;
        }
```

```
        return ret;
}

int main() {
        mat a;
        LL b;
        cin >> n >> b;

        for (int i = 1; i <= n; ++i)
                for (int j = 1; j <= n; ++j)
                        cin >> a.m[i][j];

        mat ret = qpow(a, b);

        for (int i = 1; i <= n; ++i) {
                for (int j = 1; j <= n; ++j) cout << ret.m[i][j] << ' ';
                cout << endl;
        }

        return 0;
}
```

第5章 搜索

迭代加深

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e4 + 10;
int n;

int path[N], pos;
// 有解则一路向上返回
bool dfs(int p, int depth) {
        // 迭代加深 bfs 剪枝：搜索到 depth 则停止
        if (p == depth + 1) return path[p - 1] == n;

        for (int i = 1; i < p; ++i) {
                for (int j = i; j < p; ++j) {
                        int x = path[i] + path[j];

                        // 4个 都是可行性剪枝
                        if (x <= path[p - 1]) continue; // 3. 单调递增
                        if (x > n) break; // 最后为 n, 又单调递增 -> x <= n
                        if (x + depth - p > n) break;
                        if (x * (1 << (depth - p)) < n) continue;

                        path[++pos] = x;
                        if (dfs(p + 1, depth)) return true;
                        --pos;
                }
        }

        return false;
}

void solve() {
        if (n == 1) {
                cout << 1 << endl;
                return;
        }
```

```
        path[++pos] = 1;
        for (int depth = 2; ; ++depth) {
                // 有解则停止搜索
                if (dfs(2, depth)) {
                        for (int i = 1; i <= depth; ++i) {
                                cout << path[i];
                                if (i < depth) cout << ' ';
                        }
                        cout << endl;
                        break;
                }
        }

        // 清数据
        pos = 0;
}

int main() {
        while (cin >> n, n) solve();

        return 0;
}
```

双向广搜_八数码

```
#include <bits/stdc++.h>
using namespace std;

int dx[] = {0, 0, 1, -1};
int dy[] = {1, -1, 0, 0};

string S, T = "123804765";

// 0 -> S信息，1 -> T信息
queue<string> q[2];

// st[0] : S 当前可达
// st[1] : T 当前可达
unordered_set<string> st[2];

// 对 a 扩展一层，是否有解
bool extend(int a) {
        int b = a ^ 1;
        int sz = q[a].size();

        for (int i = 1; i <= sz; ++i) {
                auto u = q[a].front(); q[a].pop();
                int pos = 0; // x 中 0 的位置
                while (u[pos] != '0') ++pos;

                // 解码
                int x = pos / 3, y = pos % 3;

                for (int j = 0; j < 4; ++j) {
                        int mx = x + dx[j], my = y + dy[j];
                        if (mx < 0 || mx > 2 || my < 0 || my > 2) continue;

                        string v = u;
                        swap(v[pos], v[3 * mx + my]); // 新pos : 3 * mx + my(编码)

                        // 刷新 dist 和 q
                        q[a].emplace(v);
                        st[a].insert(v);

                        // 有解，直接推出
```

```
                        if (st[b].count(v)) return true;
                }
        }
        return false;
}

int bfs() {
        if (S == T) return 0;

        // 初始化
        q[0].emplace(S); q[1].emplace(T);
        st[0].insert(S); st[1].insert(T);

        int step = 0; // 扩展总次数

        // 起点走不到终点 或 终点退不到起点 -> 无解
        while (q[0].size() && q[1].size()) {
                ++step;
                if (q[0].size() < q[1].size()) {
                        if (extend(0)) return step;
                }
                else {
                        if (extend(1)) return step;
                }
        }

        return -1;
}

int main() {
        cin >> S;
        int ret = bfs();
        cout << ret << endl;

        return 0;
}
```

折半搜索

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;
const int N = 2e6, INF = 0x3f3f3f3f;
LL a[100], n; LL k;
LL a1[N], a2[N];
int p1, p2;
LL path;

// 枚举[1, m], 所有可能
void ldfs(int p, int end) {
    if (p == end + 1) {
        a1[++p1] = path;
        return;
    }

    ldfs(p + 1, end);
    path += a[p];
    ldfs(p + 1, end);
    path -= a[p];
}

// 枚举[m + 1, n] 解空间
void rdfs(int p, int end) {
    if (p == end + 1) {
        a2[++p2] = path;
        return;
    }

    rdfs(p + 1, end);
    path += a[p];
    rdfs(p + 1, end);
    path -= a[p];
}

int main() {
    cin >> n >> k;
    for (int i = 1; i <= n; ++i) cin >> a[i];
    int m = n >> 1;
    ldfs(1, m);
```

```
    rdfs(m + 1, n);

    // 拼凑解空间（用双指针，二分，哈希表等暴力优化算法）
    sort(a1 + 1, a1 + p1 + 1, greater<LL>());
    sort(a2 + 1, a2 + p2 + 1);

    LL ret = 0;
    for (int i = 1, j = 1; i <= p1; ++i) {
        while(j <= p2 && a1[i] + a2[j] <= k) ++j;
        ret += j - 1;
    }
    cout << ret << endl;

    return 0;
}
```

终章 附录

关同步

```
#include <bits/stdc++.h>
using namespace std;

#define endl '\n'

const int N = 2e5 + 10, INF = 0x3f3f3f3f;


int main() {
        ios::sync_with_stdio(false); cin.tie(nullptr);



        return 0;
}
```

快速读写

```
#include <bits/stdc++.h>

inline int read() {
        int ret = 0;
        char ch = getchar();
        int flag = 1; // 标记正负

        // 跳过非数字字符
        while (ch < '0' || ch > '9') {
                if (ch == '-') flag = -1; // 处理负号
                ch = getchar();
        }

        while (ch >= '0' && ch <= '9') {
                ret = ret * 10 + ch - '0'; // 秦九韶
                ch = getchar();
        }

        return ret * flag;
}

inline void print(int x) {
        // 处理负号
        if (x < 0) {
                putchar('-');
                x = -x;
        }
        // 先处理前面的位
        if (x / 10) print(x / 10);

        // 处理个位
        putchar('0' + x % 10);
}
```

快速幂

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

LL a, b, MOD;

LL qpow(LL a, LL b) {
        LL ret = 1;
        while (b) {
                if (b & 1) ret = ret * a % MOD;

                a = a * a % MOD; b >>= 1;
        }
        return ret;
}

int main() {
        cin >> a >> b >> MOD;
        printf("%d^%d mod %d=%d", a, b, MOD, qpow(a, b));

        return 0;
}
```

龟速乘

```
#include <bits/stdc++.h>
using namespace std;
using LL = long long;

LL a, b, MOD;

LL qmul(LL a, LL b) {
        LL ret = 0;
        while (b) {
                if (b & 1) ret = (ret + a) % MOD;

                a = (a << 1) % MOD; b >>= 1;
        }
        // ret = a * b % MOD
        return ret;
}

int main() {
        cin >> a >> b >> MOD;
        cout << qmul(a, b) << endl;

        return 0;
}
```

离散化（二分）

```
#include <bits/stdc++.h>
using namespace std;

const int N = 1e5 + 10;

int disc[N], pos; // 排序去重后的数组，辅助离散化
// x 是第 id(x) 大的（保证 x 必须存在）
int id(int x, int l = 1, int r = pos) {
        while (l < r) {
                int m = l + r >> 1;
                if (x > disc[m]) l = m + 1;
                else r = m;
        }
        return l;
}

int a[N], n;

int main() {
        cin >> n;
        for (int i = 1; i <= n; ++i) cin >> a[i];

        // 离散化
        memcpy(disc, a, sizeof a); pos = n;
        sort(disc + 1, disc + n + 1);
        pos = unique(disc + 1, disc + n + 1) - (disc + 1);




        return 0;
}
```

GCD

```
#include <bits/stdc++.h>
using namespace std;

int gcd(int x, int y) {
    return y ? gcd(y, x % y) : x;
}

int main() {
    int ret = 0;
    int x;
    while (cin >> x) ret = gcd(ret, x);
    cout << ret;

    return 0;
}
```

枚举子状态

```
#include <bits/stdc++.h>
using namespace std;

void func(int k) {// 总位数
        for (int x = 0; x < (1 << k); ++x) // 枚举所有状态
        {
                // 枚举 x 的所有子状态 y
                for (int y = x; ; y = (y - 1) & x) // 正常减一，然后 & x 消除多借的位
                {

                        if (y == 0) break;
                }
        }
}
```